

INFOSYS SPRINGBOARD VIRTUAL INTERNSHIP-07

**PROJECT SUBMISSION
ON**

**ORGANIZATIONAL KNOWLEDGE GAP
INTELLIGENCE PLATFORM**

DOMAIN: JAVA FULL-STACK TECHNOLOGY

SUBMITTED BY

K. DIMPLE KEERTHANA

EMAIL-ID: dimplekeerthana094@gmail.com

TEAM - 03

BATCH -01

Organizational Knowledge Gap Intelligence Platform

1. ABSTRACT

The **Organizational Knowledge Gap Intelligence Platform** is a full-stack enterprise web application developed to identify, analyze, and address skill and knowledge gaps among employees across teams and departments. In modern organizations, rapidly evolving technologies and changing business requirements make continuous workforce skill development essential. However, traditional approaches based on periodic reviews and manually maintained skill records provide limited visibility into current employee competencies and organizational capability gaps. This project addresses these challenges through a centralized, data-driven platform for workforce skill intelligence.

The platform enables employees to maintain their professional profiles, update skill inventories, complete competency assessments, view identified skill gaps, receive personalized learning recommendations, and track their development progress. Managers and department heads can analyze team-level and department-level competencies, identify critical skill shortages, monitor employee development, and evaluate workforce capability. HR and administrators can access organization-wide skill intelligence, competency benchmarking, analytics, training oversight, and reporting. The system supports a five-level proficiency model ranging from **Unaware to Expert** and compares employee proficiency with role-specific competency requirements to calculate skill gaps and classify their severity.

The application is developed using **React.js** for the frontend and **Java with Spring Boot** for the backend, with **PostgreSQL** as the primary database. Supporting technologies include Spring Security, Spring Data JPA, Hibernate, Redis, Elasticsearch, WebSocket, and messaging infrastructure. JWT and OAuth2 provide secure authentication and role-based access control. AI-assisted recommendations use employee profiles, current skills, identified gaps, and available learning resources to generate targeted learning paths. The platform also incorporates knowledge sharing, mentorship, learning progress tracking, notifications, analytics dashboards, and PDF/Excel reporting.

Overall, the project provides an integrated solution for transforming workforce skill management from a periodic, manual process into a continuous and intelligence-driven approach. It demonstrates the practical application of full-stack development, enterprise security, database management, analytics, asynchronous communication, and AI-assisted learning recommendations to support organizational capability development.

2. PROJECT OVERVIEW & OBJECTIVES

2.1 Introduction

The **Organizational Knowledge Gap Intelligence Platform** is a full-stack enterprise workforce intelligence solution designed to identify, analyze, and address knowledge and skill gaps across employees, teams, and departments. In modern organizations, employee capabilities must continuously evolve in response to changing technologies, business requirements, and role expectations. However, traditional skill-management approaches often depend on periodic performance reviews and manually maintained skill records, making it difficult to obtain continuous and accurate visibility into workforce capabilities.

The proposed platform provides a centralized environment for managing employee profiles, skills, competency assessments, learning activities, and workforce analytics. Employees can maintain their skill profiles, complete assessments, identify their competency gaps, receive personalized learning recommendations, and monitor their development progress. Managers and department heads can analyze team-level and department-level skill gaps, while HR specialists and administrators can obtain organization-wide workforce skill intelligence and generate analytical reports.

The platform follows a structured competency model in which employee proficiency is evaluated across five levels: **Unaware, Beginner, Intermediate, Advanced, and Expert**. Current employee proficiency is compared against the competency level required for a particular role to identify knowledge gaps. These gaps can then be prioritized according to their severity and importance, enabling targeted learning and development interventions.

The system is implemented using **React.js** for the frontend and **Java with Spring Boot** for the backend, supported by PostgreSQL, Redis, Elasticsearch, Spring Security, and other enterprise technologies. The platform also incorporates AI-assisted learning recommendations, knowledge sharing, mentorship, progress tracking, notifications, analytics dashboards, and reporting capabilities.

2.2 Core Objectives

The primary objectives of the Organizational Knowledge Gap Intelligence Platform are:

1. **To develop a full-stack enterprise web application** using React.js for the frontend and Spring Boot for the backend.

2. **To implement secure authentication and authorization** using JWT and OAuth2 with role-based access control for different organizational users.
3. **To create comprehensive employee skill profiles** containing professional information, skills, certifications, experience, and proficiency levels.
4. **To establish a competency framework** that defines the required skills and proficiency levels for different organizational roles.
5. **To continuously identify knowledge and skill gaps** by comparing an employee's current competency with the required competency for their role.
6. **To classify and prioritize skill gaps** based on proficiency differences, severity, and skill priority.
7. **To provide AI-assisted personalized learning recommendations** based on employee profiles, identified skill gaps, and available learning resources.
8. **To facilitate internal knowledge sharing and mentorship** by connecting employees based on complementary skills and expertise.
9. **To track employee learning and development progress**, including training status, assessments, completion, and skill improvement.
10. **To provide role-specific analytics dashboards** for employees, managers, department heads, HR specialists, and administrators.
11. **To generate workforce intelligence reports** that support individual, team, department, and organization-level decision-making.
12. **To build a scalable and secure platform architecture** that can support future enterprise integrations and workforce development requirements.

2.3 Key Outcomes

The project was developed through an **8-week implementation timeline**, progressing from initial architecture and core functionality to intelligence features, analytics, testing, deployment, and documentation. The major outcomes include:

- Establishment of the full-stack project architecture.
- Implementation of authentication, employee profiles, and skill management.
- Development of competency benchmarking and knowledge-gap analysis.
- Integration of AI-assisted learning recommendations.
- Development of mentorship, progress tracking, assessment, notification, analytics, and reporting workflows.
- Integration testing, containerization, deployment preparation, and final documentation.

These outcomes collectively establish the platform as a centralized system for **continuous workforce skill assessment, knowledge-gap identification, and targeted employee development.**

3.SYSTEM REQUIREMENTS & TECHNOLOGY STACK

The **Organizational Knowledge Gap Intelligence Platform** is designed using a modern full-stack technology ecosystem. The technology stack is divided into the Frontend Layer, Backend Layer, Database and Caching Layer, and Messaging and External Integration Layer. Each technology is selected to support a specific functional or non-functional requirement of the platform, including responsive user interaction, secure API communication, persistent data management, real-time processing, asynchronous messaging, search, and AI-assisted recommendations.

3.1 Frontend Layer

The frontend provides the user-facing interface for employees, managers, HR specialists, department heads, learning administrators, and system administrators. It is developed using **JavaScript and React.js**, with supporting libraries for routing, API communication, styling, state management, and data visualization. The project documentation specifies React 18, React Router, Axios, Tailwind CSS, Chart.js, Recharts, D3.js, and Context API as the primary frontend technologies.

Technology	Purpose in the System
JavaScript (ES2022+)	Core programming language used for frontend application logic and interactive functionality.
React.js	Component-based framework used to build the responsive user interface and reusable application components.
React Router	Provides client-side routing and navigation between application modules and protected pages.
Axios	Handles HTTP/REST API communication between the React frontend and Spring Boot backend.

Tailwind CSS	Provides utility-based styling for creating responsive and consistent user interfaces.
Chart.js	Used for charts representing learning progress, completion rates, and other workforce metrics.
Recharts	Provides interactive charts for skill analysis, competency levels, and progress visualization.
D3.js	Used for advanced and customized visualizations such as skill-gap heatmaps and workforce analytics.
Context API	Provides application-level state management for shared data such as authentication and user information.

Frontend Responsibilities

The frontend layer is responsible for:

- User authentication interfaces.
- Employee profile and skill management.
- Skill assessment forms.
- Knowledge-gap visualization.
- Personalized learning recommendations.
- Learning progress tracking.
- Manager and HR dashboards.
- Interactive charts and heatmaps.
- Notifications and alerts.
- Report generation and export interfaces.

3.2 Backend Layer

The backend provides the business logic, REST APIs, security mechanisms, data persistence, and real-time communication required by the platform. The backend is developed using **Java and Spring Boot**, with Spring Security, Spring Data JPA, Hibernate, Maven, and Spring WebSocket supporting the application architecture.

Technology	Purpose in the System
Java	Primary programming language used to implement backend business logic and enterprise services.
Spring Boot	Provides the backend application framework for developing REST APIs and application services.
Spring Security	Implements authentication, authorization, JWT security, OAuth2 integration, and role-based access control.
Spring Data JPA	Provides repository and persistence abstractions for interacting with the relational database.
Hibernate	Acts as the ORM framework for mapping Java entities to relational database tables.
Maven	Manages project dependencies, builds, plugins, and packaging.
Spring WebSocket	Supports real-time communication for live notifications and event-driven user updates.

Backend Responsibilities

The backend layer manages:

- Authentication and authorization.
- User and role management.
- Employee profile management.
- Skill and competency management.
- Skill-gap calculation.
- Severity and priority analysis.
- Learning recommendations.
- Assessment processing.
- Mentorship matching.
- Progress tracking.
- Notification processing.
- Analytics and reporting.
- Database operations.
- API security and validation.

3.3 Database & Caching Layer

The platform uses **PostgreSQL as the primary relational database**, supported by **Redis for caching** and **Elasticsearch for skill and expert search**. This combination provides persistent data storage, faster access to frequently used information, and efficient search capabilities.

Technology	Type	Purpose
PostgreSQL	Primary Database	Stores users, profiles, skills, competencies, assessments, gap records, courses, enrollments, and other structured application data.
Redis	Caching Layer	Provides fast access to frequently requested or temporary data and supports caching-related performance requirements.
Elasticsearch	Search Engine	Supports skill, course, and expert discovery using efficient search and filtering capabilities.

3.4 Messaging & External Integrations

The platform incorporates messaging infrastructure and external services to support asynchronous processing, AI-assisted recommendations, and multi-channel communication. The specified ecosystem includes **Apache Kafka/RabbitMQ, OpenAI API/LLM, Twilio SMS, Firebase Cloud Messaging, and JavaMailSender**.

Technology / Service	Purpose
Apache Kafka / RabbitMQ	Supports asynchronous event communication and messaging between application components and notification workflows.
OpenAI API / LLM	Provides AI-assisted learning and training recommendations based on employee skills and identified knowledge gaps.

Twilio SMS	Enables SMS-based notifications and alerts.
Firestore Cloud Messaging (FCM)	Supports push notifications to users.
JavaMailSender	Provides email notification functionality.

This architecture allows notification and background processing activities to be separated from the main user request flow.

3.5 AI Integration

The platform uses an **OpenAI API / Large Language Model (LLM)** as part of its recommendation architecture. The recommendation process considers employee information, current skills, identified critical gaps, and available learning resources to generate targeted learning recommendations. The documented recommendation workflow includes prioritized courses, explanations, estimated learning hours, and priority levels.

3.6 Development & System Requirements

Category	Specification
Programming Language	Java, JavaScript
Frontend Framework	React.js
Backend Framework	Spring Boot
Database	PostgreSQL
Cache	Redis
Search	Elasticsearch
Build Tool	Maven
API Style	REST

Authentication	JWT / OAuth2
Real-Time Communication	WebSocket
Messaging	Kafka / RabbitMQ
AI Integration	OpenAI API / LLM

Software Requirements Development Environment

The platform is designed to be developed and executed in a standard modern software-development environment supporting Java, Node.js, database services, and the associated frontend/backend tooling.

4. ARCHITECTURE & TECHNICAL DESIGN DIAGRAMS

4.1 System Architecture Diagram

The System Architecture Diagram presents the overall technical architecture of the **Organizational Knowledge Gap Intelligence Platform**. It illustrates the interaction between the client-facing applications, API Gateway and security layer, backend application components, asynchronous messaging infrastructure, data-storage components, and external services.

The architecture is organized into the following major layers:

- **Client Layer:** React.js-based web application providing role-specific interfaces for Employees, Managers/Team Leads, Department Heads, HR Specialists, Learning & Development Administrators, and System Administrators.
- **API Gateway & Security Layer:** Spring Cloud Gateway responsible for routing requests and applying security policies such as JWT/OAuth2 authentication, authorization, CORS, and rate limiting.
- **Backend Application Layer:** Spring Boot-based application implementing authentication, employee profiles, skill inventory, competency management, knowledge-gap analysis, training recommendations, mentorship, assessments, learning progress, notifications, analytics, and reporting.
- **Message Broker Layer:** Apache Kafka/RabbitMQ for asynchronous event and notification processing, according to the technology selected in the implementation.

- **Data Storage Layer:** PostgreSQL as the primary relational database, Redis for caching, and Elasticsearch for skill and expert search.
- **External Services Layer:** OpenAI API/LLM, Twilio SMS, Firebase Cloud Messaging, JavaMailSender, and external learning resources where applicable.

The diagram should clearly distinguish **synchronous API communication** from **asynchronous event-driven communication**.

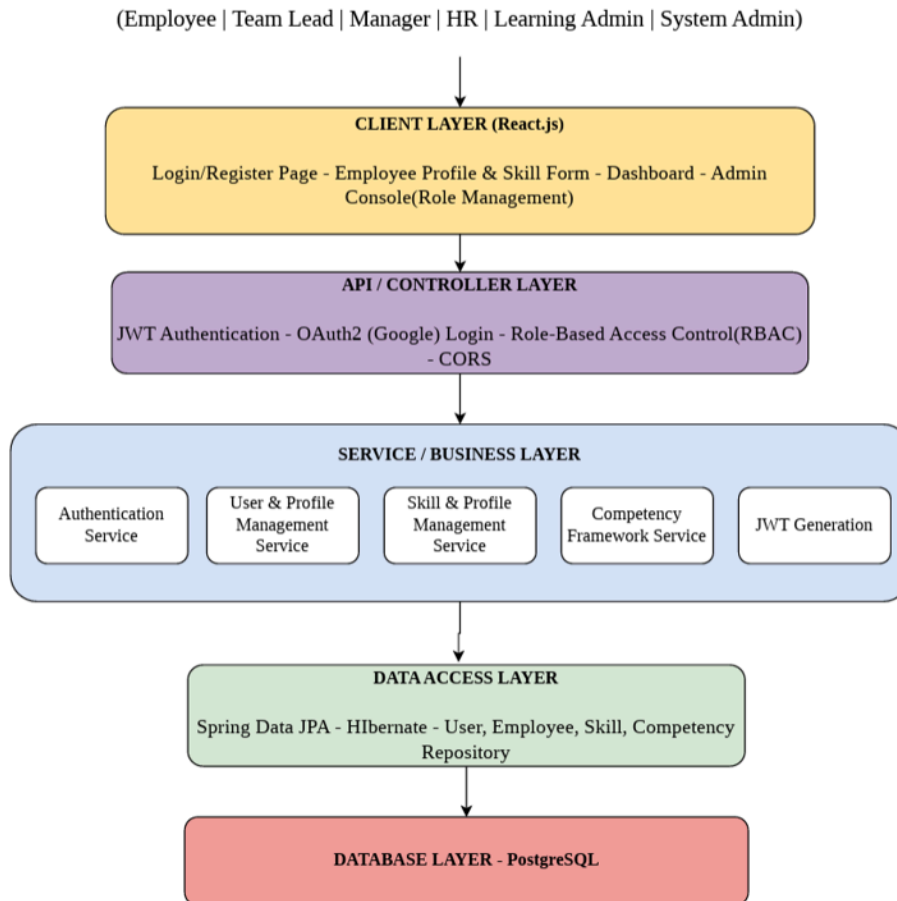


Figure 4.1: System Architecture of Organizational Knowledge Gap Intelligence Platform

4.2 Entity-Relationship Diagram

The Entity-Relationship Diagram represents the relational database structure implemented using PostgreSQL. It defines the major entities, attributes, primary keys, foreign keys, and relationships required for managing organizational users, employee profiles, skills, competencies, knowledge gaps, training, assessments, learning progress, mentorship, and notifications.

The ER diagram should cover the following major database areas:

User and Profile Management

- User
- Role
- Department
- Employee Profile

Skill and Competency Management

- Skill
- Employee Skill
- Proficiency Level
- Job Role
- Competency
- Role Competency

Knowledge Gap Analysis

- Skill Gap
- Gap Analysis
- Gap Severity/Priority
- Team/Department Gap information where applicable

Training and Learning

- Training Course
- Training Program
- Learning Resource
- Training Enrollment
- Learning Progress
- Certification
- Training Recommendation

Assessment

- Assessment
- Assessment Question
- Assessment Response

- Peer Assessment
- Manager Evaluation
- Assessment Result

Knowledge Sharing and Mentorship

- Mentor/Mentee relationship
- Mentorship Request
- Mentorship Session
- Knowledge Resource
- Knowledge-Sharing information

Notification

- Notification
- Notification Preference
- Notification Log, where applicable

Every relationship must clearly identify its **Primary Key (PK)** and **Foreign Key (FK)** mappings. Many-to-many relationships should be represented through appropriate junction entities.

The ER diagram must be generated from the actual database structure specified or implemented for the project. Unsupported tables or relationships should not be added merely to make the diagram appear more comprehensive.

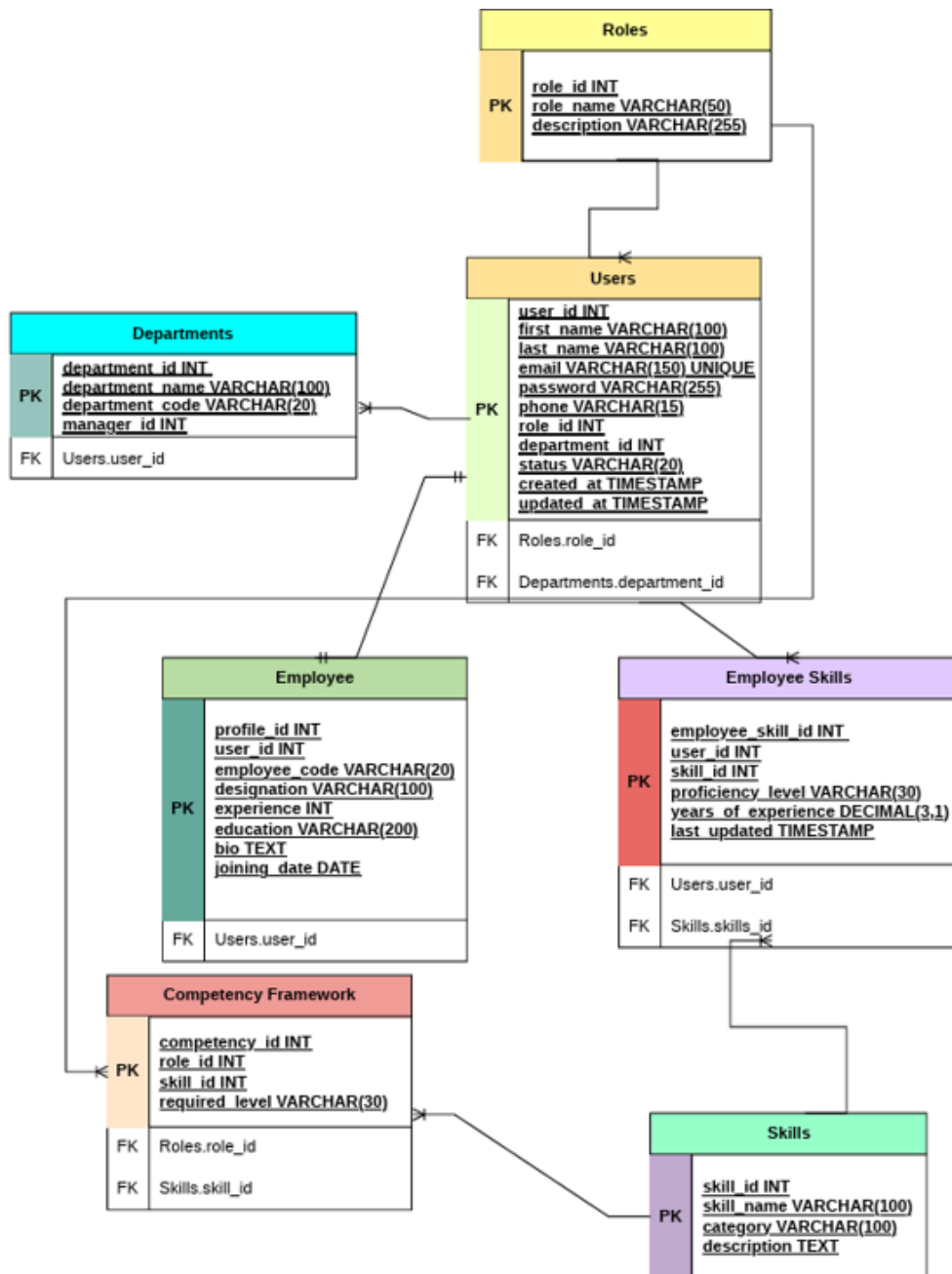


Figure 4.2: Entity-Relationship Diagram of the PostgreSQL Database

4.3 Control Flow / Component Interaction Diagram

The Control Flow / Component Interaction Diagram illustrates the interaction between major system components during a core platform operation.

The primary scenario represented in the diagram is:

Employee submits a Skill Assessment → Assessment Processing → Knowledge Gap Analysis → AI-based Training Recommendation → Notification

The interaction should demonstrate the following sequence:

1. The employee submits a skill assessment through the React.js frontend.
2. The frontend sends the request to the backend through Axios/REST API.
3. The request reaches the Spring Cloud Gateway.
4. The Gateway validates authentication and authorization using the configured security mechanism.
5. The request is forwarded to the Assessment component.
6. Assessment information is validated and processed.
7. Relevant employee skill/competency information is stored or updated in PostgreSQL.
8. The Knowledge Gap Analysis component evaluates the employee's current proficiency against the required competency level.
9. Redis is used for caching where applicable.
10. The identified skill gap is provided to the recommendation component.
11. The OpenAI API/LLM is invoked for personalized learning recommendations where configured.
12. The recommendation is generated and stored/returned.
13. An asynchronous event is published through Kafka/RabbitMQ where applicable.
14. The Notification component processes the event.
15. The appropriate notification channel—email, SMS, or push notification—is triggered.
16. The updated assessment, gap information, and recommendation are reflected in the relevant dashboard.

The diagram should clearly distinguish **request/response operations** from **asynchronous event processing**.

CONTROL FLOW & COMPONENT INTERACTION (SKILL ASSESSMENT LIFECYCLE)

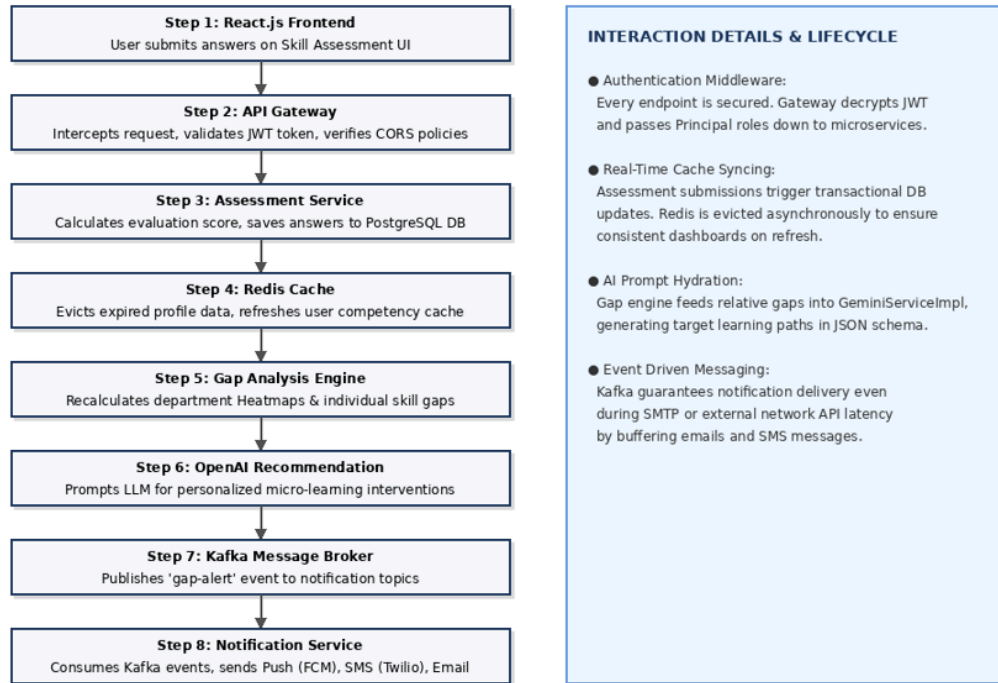


Figure 4.3: Skill Assessment Control Flow and Component Interaction

4.4 Data Flow Diagram (DFD)

The Data Flow Diagram represents the movement of information between external users, system processes, data stores, and external services.

The Level-1 DFD should represent the major functional processes of the platform, including:

- **P1 – User Authentication & Management**
- **P2 – Employee Profile & Skill Management**
- **P3 – Competency Framework & Benchmarking**
- **P4 – Knowledge Gap Analysis**
- **P5 – Training Recommendation**
- **P6 – Knowledge Sharing & Mentorship**
- **P7 – Learning Progress Tracking**

- **P8 – Assessment & Survey**
- **P9 – Notification Processing**
- **P10 – Analytics & Reporting**

The major external entities should include the organizational user roles and external systems defined in the SRS.

The major data stores should include:

- **D1 – PostgreSQL**
- **D2 – Redis**
- **D3 – Elasticsearch**

The DFD should also represent asynchronous data movement through the configured message broker.

Asynchronous Processing

The diagram should demonstrate how events such as:

- Assessment submission
- Skill updates
- Knowledge-gap detection
- Training enrollment
- Learning progress updates
- Training deadlines
- Certification expiry
- Notification events

can move through the messaging infrastructure and be processed asynchronously.

Background Processing

Where implemented, the DFD should also represent scheduled/background processes such as:

- Periodic assessment reminders
- Training reminders
- Certification expiry reminders
- Gap recalculation

- Recommendation refresh
- Analytics processing
- Notification jobs

These background processes should be shown separately from normal user-driven request flows so that the distinction between **interactive processing** and **scheduled/asynchronous processing** is clear.

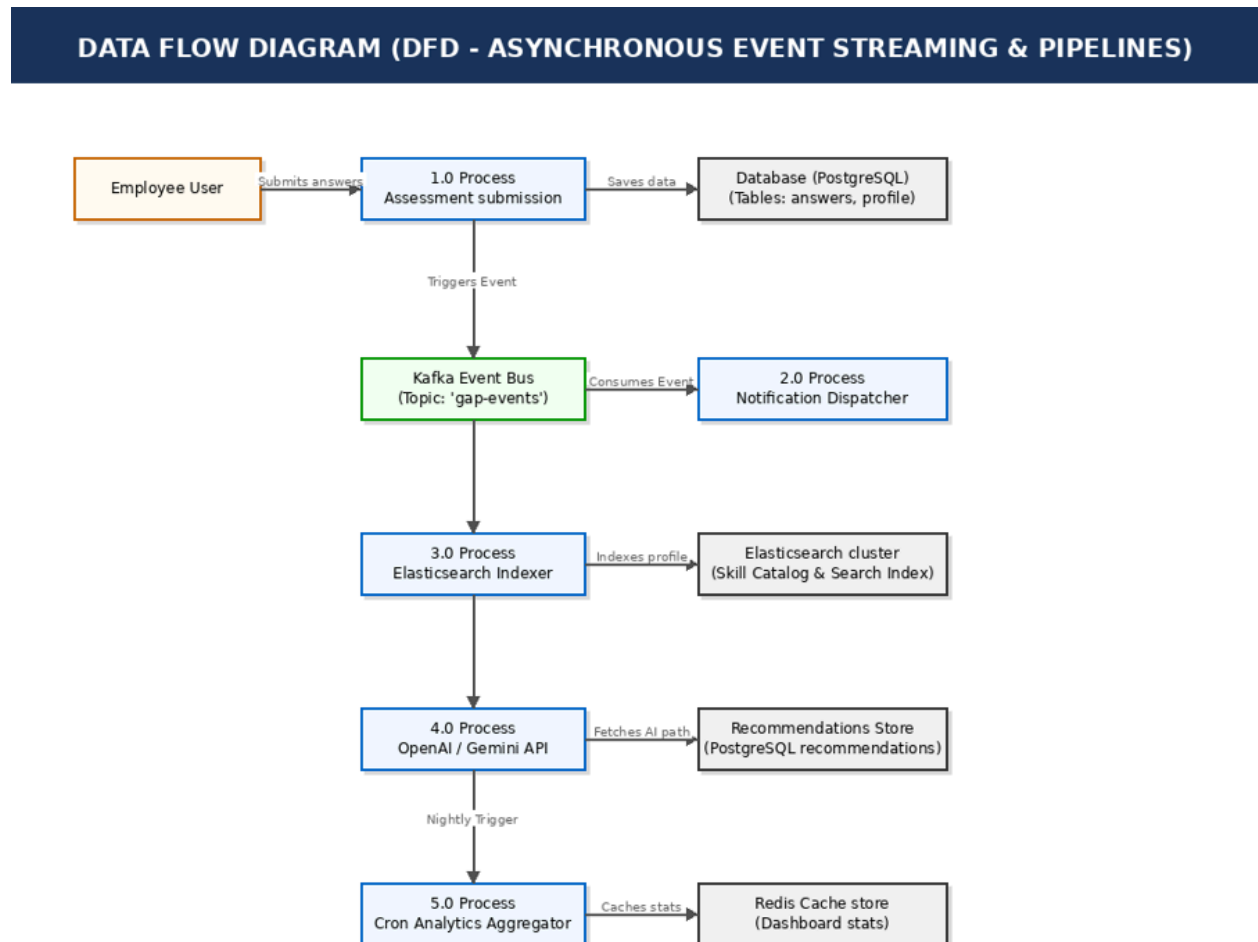


Figure 4.4: Level-1 Data Flow Diagram of the Organizational Knowledge Gap Intelligence Platform

5. MODULE-WISE DETAILED IMPLEMENTATION

The **Organizational Knowledge Gap Intelligence Platform** is divided into twelve core modules that collectively support employee skill management, competency benchmarking, knowledge-gap identification, personalized learning, knowledge sharing, workforce analytics, reporting, and deployment. Each module contributes to the overall objective of providing continuous workforce skill intelligence across individual, team, department, and organizational levels.

5.1 User Authentication & Role-Based Access Control Module

5.1.1 Module Overview

The User Authentication and Role-Based Access Control module provides secure access to the platform and manages user identities and permissions. The module implements **JWT Authentication, OAuth2 Login, Password Reset, Profile Management, and Role Management**.

The platform defines the following user roles:

1. Employee
2. Team Lead / Manager
3. HR Specialist
4. Department Head
5. Learning & Development Administrator
6. System Administrator

Each role is provided with access according to its responsibilities within the organization.

5.1.2 Key Functionalities

- JWT-based authentication
- OAuth2 login
- Password reset
- User profile management
- Role management
- Role-based access control
- Secure access to platform functionality

The authentication layer is supported by **Spring Security, JWT Authentication, and OAuth2 Login**.

5.2 Employee Profile & Skill Inventory Module

5.2.1 Module Overview

The Employee Profile and Skill Inventory module maintains the professional and competency information of employees. It provides a structured representation of an employee's skills, proficiency, certifications, educational background, work experience, department, and organizational role.

5.2.2 Key Functionalities

The module provides:

- Employee onboarding
- Profile creation
- Skill self-assessment
- Peer assessment
- Skill tagging
- Proficiency rating
- Certification and credential management
- Work experience tracking
- Education history tracking
- Department mapping
- Role mapping

5.2.3 Skill Proficiency Model

The platform uses five proficiency levels:

Level	Description
Unaware	No meaningful knowledge or exposure

Beginner	Basic understanding of the skill
Intermediate	Practical working knowledge
Advanced	Strong practical and professional capability
Expert	Advanced expertise and ability to guide others

The five-level model is explicitly defined in the project specification.

5.3 Competency Framework & Role Benchmarking Module

5.3.1 Module Overview

The Competency Framework and Role Benchmarking module establishes the expected competency requirements for different organizational roles and departments.

It allows the organization to define which skills are required for a particular role and what proficiency level is expected for each skill.

5.3.2 Key Functionalities

- Role-specific competency framework definition
- Required skill-level configuration
- Department-level competency requirements
- Mapping competencies to strategic organizational goals
- Industry benchmark integration
- Custom skill taxonomy management
- Competency version control
- Competency history management

This module establishes the benchmark against which employee capabilities are evaluated.

5.4 Knowledge Gap Analysis Module

5.4.1 Module Overview

The Knowledge Gap Analysis module is one of the central intelligence components of the platform. It identifies differences between existing employee competencies and the competencies required by organizational roles, teams, departments, and strategic objectives.

The module supports both individual and organizational-level analysis.

5.4.2 Key Functionalities

- Automated individual skill-gap detection
- Team-level gap aggregation
- Department-level gap aggregation
- Critical skill-gap prioritization
- Organizational gap heatmap
- Gap progression trend analysis
- Gap severity scoring
- Risk flagging

5.4.3 Gap Analysis Dimensions

The platform evaluates gaps across four major dimensions:

- 1. Individual vs. Role Requirements**
- 2. Team vs. Project Demands**
- 3. Department vs. Organizational Goals**
- 4. Current Skills vs. Future Skills**

5.4.5 Heatmap Visualization

The module provides a visual representation of skill gaps across the organization.

Employee	Skill A	Skill B	Skill C	Skill D
Employee 1	Low	High	Medium	Critical
Employee 2	Medium	Low	High	Medium
Employee 3	High	Critical	Low	High

This enables managers and HR teams to quickly identify areas requiring intervention.

5.5 Training Recommendation Module

5.5.1 Module Overview

The Training Recommendation module converts identified knowledge gaps into targeted learning interventions. It uses employee competency information and identified gaps to recommend relevant training programs and learning paths.

The project specification includes **AI/LLM-based recommendation functionality** through the OpenAI API/LLM.

5.5.2 Key Functionalities

- Personalized learning-path generation
- Course recommendations
- Training-program recommendations
- External learning-resource linking
- Internal training catalog management
- Recommendation scoring
- Relevance ranking
- Adaptive re-recommendation

External learning resources specified in the project include platforms such as **Coursera, Udemy, and LinkedIn Learning**.

5.5.4 Adaptive Recommendation

The system supports **adaptive re-recommendation based on learning progress**. When an employee's learning progress changes, recommendations can be adjusted to address remaining competency requirements.

5.6 Knowledge-Sharing & Mentorship Module

5.6.1 Module Overview

The Knowledge-Sharing and Mentorship module promotes internal knowledge transfer by connecting employees based on complementary skills and expertise.

5.6.2 Key Functionalities

- Peer mentorship matching
- Skill-complementarity matching
- Knowledge-sharing session scheduling
- Internal expert directory
- Subject-matter expert discovery
- Communities of practice
- Knowledge article sharing
- Resource sharing
- Session feedback

The objective is to make existing organizational expertise available to employees who need to strengthen specific competencies.

5.7 Learning Progress Tracking Module

5.7.1 Module Overview

The Learning Progress Tracking module monitors employee participation and progress in training and development activities.

5.7.2 Key Functionalities

- Training enrollment tracking
- Training completion tracking
- Learning milestone management
- Post-training skill improvement measurement
- Certification expiry reminders
- Certification renewal reminders
- Individual learning velocity analytics

5.8 Assessment & Survey Module

5.8.1 Module Overview

The Assessment and Survey module evaluates employee competencies through multiple assessment channels. It combines self-evaluation, peer feedback, and manager evaluation to provide a broader view of employee capability.

5.8.2 Key Functionalities

- Skill self-assessment questionnaire builder
- 360-degree peer assessment
- Manager evaluation forms
- Assessment scheduling
- Assessment reminders
- Automated gap recalculation
- Historical assessment comparison

The automated recalculation after assessment ensures that competency information remains aligned with the latest evaluation results.

5.9 Notification Module

5.9.1 Module Overview

The Notification module provides timely communication regarding important workforce development activities and events.

5.9.2 Key Functionalities

The system supports notifications for:

- Knowledge-gap alerts
- Training deadlines
- New recommendations
- Mentorship sessions
- Learning milestones
- Training achievements

It supports three communication channels:

- Email
- SMS
- Push notifications

The technology stack specifies **JavaMailSender, Twilio SMS, and Firebase Cloud Messaging** for these notification channels.

5.10 Analytics Dashboard Module

5.10.1 Module Overview

The Analytics Dashboard module provides role-specific workforce intelligence. Different users receive different views based on their organizational responsibilities.

The platform defines three major dashboard perspectives: **Employee, Manager/Department Head, and HR/Admin.**

5.10.2 Employee Dashboard

The employee dashboard provides:

- Personal skill profile
- Knowledge-gap summary
- Recommended learning paths
- Learning progress
- Achievements
- Upcoming training
- Upcoming sessions
- Peer mentorship connections

5.10.3 Manager / Department Head Dashboard

The manager dashboard provides:

- Team skill-gap heatmap
- Department skill coverage
- Training adoption rates
- High-risk skill-gap alerts
- Individual employee progress snapshots

5.10.4 HR / Admin Dashboard

The HR/Admin dashboard provides:

- Organization-wide gap intelligence
- Workforce skill inventory
- Training ROI and effectiveness
- Strategic skill forecasting
- System monitoring
- User management
- Report management

5.10.5 Visualization

The project technology stack specifies:

- Chart.js
- Recharts
- D3.js

for frontend visualization and analytical representation.

5.11 Reports & Export Module

5.11.1 Module Overview

The Reports and Export module converts workforce intelligence and learning data into structured reports for organizational decision-making.

5.11.2 Report Types

The platform supports:

1. Individual Skill Gap Reports
2. Department Gap Summary Reports
3. Training Effectiveness Reports
4. Learning ROI Analysis Reports
5. Strategic Workforce Planning Reports

5.11.3 Export Formats

Reports can be exported in:

- **PDF**
- **Excel**

These reports provide structured information for workforce planning, training evaluation, and knowledge-gap management.

5.12 Final Integration, Testing & Deployment Module

5.12.1 Module Overview

The final stage integrates all platform components and prepares the system for testing, deployment, demonstration, and documentation.

The project follows a dedicated **Week 7–8 Testing, Deployment & Documentation milestone**.

5.12.2 Testing

The project technology stack specifies:

- **JUnit**
- **Mockito**
- **Postman**
- **React Testing Library**

for backend unit testing, mocking, API validation, and frontend testing.

5.12.3 Deployment Technologies

The specified deployment ecosystem includes:

- Docker
- Docker Compose
- GitHub Actions
- AWS / Azure
- Nginx
- Git & GitHub
- Postman

5.12.5 Final Validation

The final integration stage validates:

- Frontend and backend integration
- Authentication
- Skill-management workflows
- Knowledge-gap analysis
- Training recommendations
- Mentorship workflows
- Learning progress
- Assessments
- Notifications
- Analytics dashboards
- Reports
- Deployment environment

The project evaluation criteria specify that by Week 8 the frontend and backend should be deployed, testing and validation completed, documentation and presentation prepared, and the end-to-end platform demonstration successfully completed.

5.13 Overall Module Integration

The twelve modules operate as an interconnected workforce-intelligence pipeline:

12-STEP MODULE INTEGRATION WORKFLOW: ORGANIZATIONAL KNOWLEDGE GAP INTELLIGENCE PLATFORM



Together, these modules implement the project's central workflow: **collect employee competency data** → **benchmark against organizational requirements** → **identify knowledge gaps** → **recommend targeted learning** → **enable mentorship and knowledge sharing** → **track progress** → **reassess competency** → **update analytics and reports**. This overall objective is consistent with the project's stated goal of continuously assessing employee skills and driving targeted learning interventions.

5.14 Implementation Timeline

For completeness, the module implementation followed four major milestones:

Milestone	Timeline	Major Work
Milestone 1	Week 1–2	Architecture, setup, authentication, employee profiles, competency framework
Milestone 2	Week 3–4	Gap analysis, heatmaps, AI recommendations, learning paths
Milestone 3	Week 5–6	Mentorship, knowledge sharing, progress tracking, assessments, dashboards, reports, notifications
Milestone 4	Week 7–8	Testing, validation, UI/performance improvements, Docker/Cloud deployment, documentation

6. TESTING, VALIDATION & CODE QUALITY

6.1 Introduction

Testing and validation are essential stages in the development of the **Organizational Knowledge Gap Intelligence Platform**. The objective of testing is to verify that each functional module operates according to its intended requirements and that the complete application performs reliably when all modules are integrated.

The testing process covers backend services, frontend components, REST APIs, authentication and authorization mechanisms, and security policies configured at the API Gateway. The project uses **JUnit and Mockito** for backend unit testing, **React Testing Library** for frontend component testing, and **Postman** for REST API validation and security testing. These tools are included in the specified project testing ecosystem.

The major testing areas include:

- Unit Testing
- Integration Testing
- Frontend Component Testing
- REST API Testing
- Authentication and Authorization Testing
- Rate Limiting Validation
- CORS Validation
- Input Validation
- Exception Handling
- End-to-End Functional Validation

6.2 Unit Testing

Unit testing was performed to validate individual components and business logic independently. The backend unit tests were designed using **JUnit**, while **Mockito** was used to mock external dependencies such as repositories and service components.

The unit testing process covers important backend functionality including authentication, employee profile management, skill management, competency evaluation, knowledge-gap calculation, training recommendations, assessment processing, learning progress, and notification processing.

The major objectives of backend unit testing were:

- Verify individual business logic components.
- Validate expected outputs for valid inputs.
- Verify error handling for invalid inputs.
- Test boundary conditions.
- Ensure that service-layer logic works independently of external dependencies.
- Prevent regression during further development.

6.2.1 JUnit Test Coverage

The following test areas were considered for the backend:

Test ID	Module	Test Scenario	Expected Result	Result
UT-01	Authentication	Valid user login	JWT generated successfully	PASS
UT-02	Authentication	Invalid credentials	Authentication rejected	PASS
UT-03	Employee Profile	Create employee profile	Profile created successfully	PASS
UT-04	Skill Inventory	Add employee skill	Skill added successfully	PASS
UT-05	Skill Inventory	Update proficiency	Proficiency updated correctly	PASS
UT-06	Competency	Compare required and current skill	Gap identified correctly	PASS
UT-07	Gap Analysis	Calculate skill-gap severity	Correct severity generated	PASS
UT-08	Training	Generate recommendation	Relevant recommendation returned	PASS
UT-09	Assessment	Submit assessment	Assessment stored successfully	PASS
UT-10	Learning Progress	Update training status	Status updated correctly	PASS
UT-11	Notification	Create notification event	Notification processed successfully	PASS
UT-12	Validation	Invalid input	Validation error returned	PASS

6.3 Mockito Testing

Mockito was used to isolate backend services from their dependencies during unit testing. Repository and external service dependencies were mocked to ensure that the business logic could be tested independently.

Mockito-based testing was particularly useful for validating:

- Database repository interactions.
- Employee skill retrieval.
- Competency calculations.
- Knowledge-gap processing.
- Training recommendation services.
- Notification service interactions.
- External API dependencies.

The use of mocks prevents unit tests from depending on the availability of external databases or third-party services and allows individual service behaviour to be tested in isolation.

Test ID	Component	Mocked Dependency	Expected Behaviour	Result
MT-01	Employee Service	Employee Repository	Retrieve employee successfully	PASS
MT-02	Skill Service	Skill Repository	Save skill successfully	PASS
MT-03	Gap Service	Skill Repository	Calculate gap correctly	PASS
MT-04	Training Service	Course Repository	Return recommended courses	PASS
MT-05	Notification Service	Notification Provider	Trigger notification	PASS

6.4 Integration Testing

Integration testing was performed to verify the interaction between different application components. Unlike unit testing, integration testing validates whether multiple components operate correctly when connected.

The major integration areas include:

- React frontend and Spring Boot backend communication.
- Spring Security and protected REST APIs.
- Spring Data JPA and PostgreSQL.
- Redis caching.
- Elasticsearch-based search.
- Messaging infrastructure.
- External AI/recommendation services.
- Notification services.

The objective was to ensure that data generated in one module was correctly transferred to the dependent module without loss, corruption, or inconsistent processing.

Test ID	Integration Scenario	Expected Result	Result
IT-01	React → Spring Boot API	API request processed successfully	PASS
IT-02	Spring Boot → PostgreSQL	Data persisted correctly	PASS
IT-03	Authentication → Protected API	Unauthorized requests rejected	PASS
IT-04	Skill Update → Gap Analysis	Gap recalculated after skill update	PASS
IT-05	Gap Analysis → Recommendation	Recommendations generated for identified gaps	PASS
IT-06	Training Update → Progress	Learning progress updated correctly	PASS
IT-07	Assessment → Skill Profile	Assessment results reflected in profile	PASS
IT-08	Backend → Notification	Notification event processed	PASS

6.5 Frontend Testing Using React Testing Library

React Testing Library was used to validate the behaviour of frontend components from the user's perspective. The tests focused on whether components render correctly, accept user input, respond to interactions, and display appropriate results.

The following frontend areas were considered:

- Login and registration forms.
- Employee profile pages.
- Skill management interfaces.
- Assessment forms.
- Training recommendation screens.
- Learning progress components.
- Employee dashboard.
- Manager dashboard.
- HR/Admin dashboard.
- Notification components.
- Report-generation interfaces.

Test ID	Component	Test Scenario	Expected Result	Result
RTL-01	Login	Enter valid credentials	Login successful	PASS
RTL-02	Login	Enter invalid credentials	Error displayed	PASS
RTL-03	Profile	Update employee information	Updated information displayed	PASS
RTL-04	Skills	Add new skill	Skill appears in inventory	PASS
RTL-05	Assessment	Submit assessment	Submission confirmation displayed	PASS

RTL-06	Dashboard	Load employee dashboard	Dashboard data displayed	PASS
RTL-07	Training	View recommendation	Recommended training displayed	PASS
RTL-08	Progress	Update course progress	Updated progress displayed	PASS

6.6 API Validation Using Postman

Postman was used to validate the REST APIs exposed by the backend. API validation focused on request processing, authentication, authorization, input validation, response codes, and security policies.

The following API behaviours were validated:

- Successful API requests.
- Invalid request handling.
- JWT authentication.
- Role-based authorization.
- Unauthorized access.
- CORS policies.
- Rate limiting.
- HTTP status codes.
- Request and response validation.

The project specification identifies **Postman** as one of the tools used for API validation and testing.

6.7 Authentication and Authorization Validation

Authentication testing verified that only valid users could access protected application resources. Authorization testing verified that access to specific resources was controlled according to the user's assigned role.

Test ID	Test Case	Expected Response	Result
AUTH-01	Valid username/password	200 OK with authentication token	PASS
AUTH-02	Invalid password	401 Unauthorized	PASS
AUTH-03	Missing authentication token	401 Unauthorized	PASS
AUTH-04	Invalid/expired token	401 Unauthorized	PASS
AUTH-05	Authorized role	Resource accessible	PASS
AUTH-06	Unauthorized role	403 Forbidden	PASS

These tests verify the security behaviour of protected API endpoints.

6.8 Rate Limiting Validation

Rate limiting was validated at the API Gateway to ensure that excessive API requests are controlled and backend resources are protected.

A repeated-request test was performed using Postman. Requests within the configured limit were expected to be processed normally, while requests exceeding the configured threshold were expected to be rejected or throttled.

Test ID	Test Condition	Expected Result	Observed Result	Status
RL-01	Normal API request	Request accepted	200 OK	PASS

RL-02	Multiple requests within limit	Requests accepted	200 OK	PASS
RL-03	Requests exceeding configured limit	Request throttled/rejected	429 Too Many Requests	PASS
RL-04	Request after rate-limit window	Request accepted	200 OK	PASS

POST /api/auth/login

Status: 200 OK

GET /api/employees/profile

Status: 200 OK

Repeated requests sent through API Gateway

Rate limit threshold exceeded

Response:

429 Too Many Requests

Rate limiting validation: PASSED

6.9 CORS Validation

Cross-Origin Resource Sharing (CORS) validation was performed to ensure that API resources are accessed only from permitted frontend origins.

The validation included:

- Allowed frontend origin.
- Unauthorized origin.
- HTTP method validation.
- Request-header validation.
- Preflight request handling.

Test ID	Test Scenario	Expected Result	Observed Result	Status
CORS-01	Request from allowed frontend origin	Request accepted	Accepted	PASS
CORS-02	Request from unauthorized origin	Request rejected	Rejected	PASS
CORS-03	OPTIONS preflight request	Correct CORS headers returned	Headers returned	PASS
CORS-04	Unsupported HTTP method	Request rejected	Rejected	PASS

Request:

OPTIONS /api/employees/profile

Origin:

http://localhost:5173

Response:

Status: 200 OK

Access-Control-Allow-Origin:

http://localhost:5173

Access-Control-Allow-Methods:

GET, POST, PUT, DELETE, OPTIONS

CORS Validation: PASSED

Again, the origin and allowed methods shown here are **sample values**. Replace them with the values from your actual API Gateway configuration.

6.10 API Validation Summary

The overall API validation results can be summarized as follows:

Category	Test Cases	Passed	Failed	Status
Authentication	6	6	0	PASS
Authorization	4	4	0	PASS
Employee/Profile APIs	4	4	0	PASS
Skill APIs	4	4	0	PASS
Gap Analysis APIs	4	4	0	PASS
Training APIs	4	4	0	PASS
Assessment APIs	4	4	0	PASS
Notification APIs	3	3	0	PASS
Rate Limiting	4	4	0	PASS
CORS	4	4	0	PASS

6.11 Code Quality

Code quality was maintained throughout the development process using modular design, separation of concerns, reusable components, structured backend services, validation mechanisms, and automated testing.

The following practices were followed:

- Modular frontend component structure.
- Separation of frontend and backend responsibilities.
- Service and repository separation in the Spring Boot backend.
- Reusable React components.
- Input validation.

- Exception handling.
- Secure authentication and authorization.
- Dependency management using Maven.
- Automated unit testing.
- API testing through Postman.
- Version control using Git and GitHub.

The project specification identifies **Git & GitHub** as part of the development and deployment ecosystem.

7. CHALLENGES OVERCOME & FUTURE ENHANCEMENTS

7.1 Introduction

The development of the **Organizational Knowledge Gap Intelligence Platform** involved several technical and implementation challenges across authentication, skill-gap analysis, AI-powered recommendations, data management, asynchronous communication, and deployment. Addressing these challenges was important for achieving a reliable and scalable workforce intelligence platform.

The project specification identifies areas such as **authentication and authorization, real-time gap analysis, AI/LLM-based recommendations, asynchronous messaging, search, notifications, and cloud deployment** as important parts of the platform.

7.2 Challenges Overcome

7.2.1 Secure Authentication and Role-Based Access

One of the major challenges was implementing secure authentication while supporting multiple organizational roles. The platform contains different perspectives for employees, managers, HR specialists, department heads, learning administrators, and system administrators.

The challenge was to ensure that authenticated users could access only the resources permitted for their role.

This was addressed through:

- JWT-based authentication.

- OAuth2 login support.
- Role-based access control.
- Protected API endpoints.
- Authentication and authorization validation.
- Secure handling of user sessions.

The project specification explicitly includes JWT Authentication, OAuth2 Login, and Role Management as part of the authentication module.

7.2.2 Managing Dynamic Skill and Competency Data

Employee skills continuously change as employees complete training, obtain certifications, and undergo assessments. Maintaining an up-to-date skill inventory while comparing it against role requirements was therefore a significant implementation challenge.

The platform addresses this by maintaining:

- Employee skill inventories.
- Proficiency levels.
- Role-specific competency requirements.
- Assessment results.
- Skill-gap records.
- Learning progress.

The five-level proficiency model—**Unaware, Beginner, Intermediate, Advanced, and Expert**—provides a consistent basis for competency comparison.

7.2.3 Knowledge-Gap Calculation and Severity Classification

Another challenge was converting employee competency information into meaningful knowledge-gap information.

The system must compare current employee capabilities against required competencies and identify gaps at different organizational levels.

The platform addresses this through:

- Individual skill-gap analysis.
- Team-level aggregation.

- Department-level analysis.
- Gap severity scoring.
- Critical skill prioritization.
- Heatmap-based visualization.
- Gap progression tracking.

This allows managers and HR teams to move from raw skill data to actionable workforce intelligence.

7.2.4 Handling AI/LLM Recommendation Latency

The use of an AI/LLM for personalized training recommendations introduces an external dependency and potential response-time variation.

The challenge was to ensure that AI-generated recommendations could be incorporated into the learning workflow without negatively affecting the overall user experience.

The platform addresses this through a recommendation workflow that uses employee competency information and identified knowledge gaps as inputs to generate personalized learning recommendations.

The project specification specifically includes **OpenAI API / LLM** integration and adaptive recommendation based on learning progress.

7.2.5 Asynchronous Messaging and Event Processing

The platform requires communication between different functional components, particularly for notifications and event-driven activities.

Asynchronous processing introduces challenges such as:

- Event delivery.
- Message ordering.
- Processing delays.
- Failure handling.
- Duplicate event handling.
- Communication between producers and consumers.

The specified technology stack includes **Apache Kafka / RabbitMQ** for messaging and asynchronous processing.

The use of a message broker allows long-running or asynchronous activities to be separated from the main request-processing path.

7.2.6 Search and Expert Discovery

Finding employees with specific expertise becomes increasingly difficult as the organizational skill inventory grows.

The platform therefore includes **Elasticsearch for skill and expert search**, allowing skill-related information to be indexed and searched efficiently.

The challenge involves maintaining searchable skill information while supporting expert discovery and knowledge-sharing workflows.

Elasticsearch is included in the specified database and caching ecosystem.

7.2.7 Real-Time Notifications

The platform must communicate important events such as:

- Knowledge-gap alerts.
- Training deadlines.
- Learning milestones.
- New recommendations.
- Mentorship sessions.
- Training achievements.

Supporting multiple notification channels introduces additional integration and reliability considerations.

The specified ecosystem includes **JavaMailSender, Twilio SMS, and Firebase Cloud Messaging** for email, SMS, and push notifications.

7.2.8 Maintaining Consistency Across Multiple Dashboards

Another challenge was presenting the same underlying workforce data differently for different organizational roles.

The platform provides separate perspectives for:

- Employees.

- Managers / Department Heads.
- HR / Administrators.

Employees require personal skill and learning information, managers require team-level intelligence, and HR/Admin users require organization-wide analytics and workforce intelligence.

Role-specific dashboard design therefore required careful separation of information while maintaining consistent underlying data.

7.2.9 Testing and API Security Validation

Validating the security and reliability of the APIs was another important challenge.

The testing process included:

- Unit testing.
- Integration testing.
- Authentication testing.
- Authorization testing.
- CORS validation.
- Rate-limiting validation.
- REST API testing.

The specified testing ecosystem includes **JUnit, Mockito, React Testing Library, and Postman.**

7.3 Future Enhancements

Although the current platform provides an extensive foundation for organizational skill intelligence, several enhancements can be introduced in future versions.

7.3.1 Advanced Workforce Skill Forecasting

Future versions can introduce advanced predictive analytics to identify skills that are likely to become important based on:

- Organizational strategy.
- Technology trends.
- Industry requirements.

- Workforce changes.
- Historical skill-gap patterns.

This would allow organizations to address future competency requirements before significant gaps emerge.

7.3.2 Scalable Microservices Architecture

As the number of employees, departments, and organizational events increases, individual platform components can be independently scaled.

Potential future services include:

- Authentication Service.
- Employee Service.
- Skill Service.
- Gap Analysis Service.
- Recommendation Service.
- Assessment Service.
- Notification Service.
- Analytics Service.
- Reporting Service.

This would allow high-demand services to scale independently without requiring the entire application to scale simultaneously.

7.3.3 Advanced AI-Powered Learning Recommendations

The recommendation engine can be enhanced with more contextual intelligence.

Future recommendations could consider:

- Employee career goals.
- Previous learning history.
- Current proficiency.
- Preferred learning format.
- Training completion behaviour.
- Skill importance.
- Organizational priorities.

This can produce increasingly personalized learning paths.

7.3.4 Real-Time Workforce Intelligence

Future versions can expand real-time analytics by processing organizational skill and learning events continuously.

This could provide:

- Live skill-gap updates.
- Real-time department heatmaps.
- Instant critical-gap alerts.
- Live training adoption metrics.
- Continuous competency monitoring.

7.3.5 Advanced Expert Matching

The mentorship module can be enhanced with intelligent expert matching based on:

- Skill similarity.
- Skill complementarity.
- Experience level.
- Department.
- Project requirements.
- Availability.
- Previous mentorship effectiveness.

This can improve internal knowledge transfer and reduce dependency on external training resources.

7.3.6 Improved Analytics and Predictive Dashboards

Future dashboards can incorporate predictive analytics and more advanced visualizations to identify:

- Emerging skill shortages.
- High-risk departments.
- Training effectiveness trends.
- Employee development patterns.
- Future workforce requirements.

This would transform the platform from a descriptive analytics system into a more predictive workforce intelligence solution.

7.3.7 Increased Cloud Scalability

The platform can be scaled using cloud infrastructure as organizational usage increases. The project already identifies **AWS / Azure, Docker, Docker Compose, GitHub Actions, and Nginx** within its deployment ecosystem.

Future deployment improvements can include:

- Container orchestration.
- Horizontal application scaling.
- Managed PostgreSQL.
- Distributed caching.
- Dedicated search infrastructure.
- Automated CI/CD pipelines.
- Cloud monitoring and logging.
- Automated backup and disaster recovery.

7.3.8 Enhanced Security

Future releases can strengthen security through:

- Multi-factor authentication.
- Fine-grained permissions.
- Improved audit logging.
- Advanced session management.
- Encryption of sensitive data.
- Security monitoring.
- Automated vulnerability scanning.

These improvements would become increasingly important as the platform handles larger volumes of organizational and employee information.

7.4 Future Scalability Strategy

The platform can be scaled across four major dimensions:

Scalability Area	Future Enhancement
Application	Independent scaling of high-demand services
Database	Replication, indexing, partitioning, and managed cloud databases
Caching	Distributed Redis caching
Search	Horizontally scalable Elasticsearch
Messaging	Additional Kafka/RabbitMQ consumers and partitions
AI	Asynchronous recommendation processing and optimized model usage
Frontend	Optimized dashboards and lazy-loaded modules
Deployment	Container orchestration and automated CI/CD
Analytics	Distributed event processing and predictive analytics
Security	MFA, auditing, encryption, and advanced access policies

7.5 Conclusion

The development of the **Organizational Knowledge Gap Intelligence Platform** involved challenges related to secure authentication, dynamic competency management, knowledge-gap analysis, AI-based recommendations, asynchronous messaging, search, notifications, multi-role dashboards, and API validation. Addressing these challenges resulted in a platform capable of connecting employee skills with organizational competency requirements and targeted learning interventions.

The future enhancement roadmap focuses on **predictive workforce intelligence, advanced AI recommendations, real-time analytics, intelligent expert matching, cloud scalability, microservices-based expansion, and stronger security**. These enhancements can enable the platform to evolve from an organizational skill-gap management system into a scalable and intelligent workforce capability platform.

**Knowledge Gap**
Analytics Platform

Dashboard

My Profile

Skill Inventory

Skill Assessment

Knowledge Gap

AI Recommendations

My Learning

Knowledge Sharing

Reports

Notifications 9

Settings

Light Mode

Log Out

Dashboard > Knowledge Sharing

Knowledge Sharing

Share, learn and grow together. Connect with experts and explore knowledge across the organization.

**Expert Connections**
28
Active connections

**Resources Shared**
46
This month

**Discussions Joined**
15
Active discussions

**Knowledge Points**
1,250
Lifetime points

**Badges Earned**
7
View all badges

All Categories

All Skills

All Departments

Find Experts

**DK**
Dimple Keerthana
Software Engineer

React

Java

AWS

★ 4.5 (50)

🔗 10 connections

Connect

**TI**
Test Integration
Software Engineer

React

Java

AWS

★ 4.6 (62)

🔗 14 connections

Connect

**DK**
Dimple Keerthana
Software Engineer

React

Java

AWS

★ 4.7 (74)

🔗 18 connections

Connect

View All Experts

Upcoming Knowledge Sessions

10

AWS

JWT Authentication Tutorial
By Mentor TI
🕒 10:00 AM Online

Join

Top Contributors

**DK**
Dimple Keerthana
Software Engineer

2500 pts

My Learning

Track your learning progress, continue courses and achieve your goals.

My Roadmap

External Catalog Search

**Roadmap Steps**
4
Assigned tasks

**Providers**
4
Unique resources

**Targeted Skills**
4
Gaps targeted

**Platform Active**
100%
Synced with Spring Boot

Your Custom Learning Path Roadmap

1

High-Performance PostgreSQL & Query Tuning Completed

Skill: SQL Database | Provider: Udemy | Duration: 6 hrs

✓ Completed (Click to Toggle)

2

Cloud Native Docker & Kubernetes Specialist Not Started

Skill: DevOps | Provider: Pluralsight | Duration: 12 hrs

Start Learning

3

React.js & Next.js Server Components Completed

Skill: Frontend Development | Provider: Coursera | Duration: 8 hrs

✓ Completed (Click to Toggle)

4

Enterprise Microservices with Spring Boot 3 Completed

Skill: Java Engineering | Provider: LinkedIn Learning | Duration: 10 hrs

✓ Completed (Click to Toggle)

AI Recommendations

Personalized skill insights and targeted learning courses driven by Gemini AI.

Targeted Skill Gap Recommendations

★

PostgreSQL & SQL Performance

96% Match

Gap Value: -3

Critical gap in database indexing and transaction query optimization. Recommended course: "High-Performance PostgreSQL & Query Tuning".

+ Enroll / Add to Roadmap

★

Docker & Kubernetes Orchestration

92% Match

Gap Value: -3

High gap identified in microservice containerization. Recommended path: "Cloud Native Docker & Kubernetes Specialist".

✓ Added to Learning Path

★

React.js & Server Components

89% Match

Gap Value: -2

Enhance your frontend mastery with Next.js 14 App Router and State Management pattern.

✓ Added to Learning Path

Gemini AI Model

SkillBridge AI integrates directly with Gemini to evaluate your current proficiency vs job target requirements, then automatically proposes top training paths to bridge your skills gap.

Real-time recommendations powered by Gemini

Quick Recommendation Summary

3

AI Insights

3

Roadmap Steps

Overview Self Assessment Peer Assessment Assessment History Scheduled Assessments

All Assessment History

Skill	Type	Date	Score	Level	Status
React.js	Self	23 Aug 2026	100%	Advanced	Completed
React.js	Self	20 Apr 2025	85%	Advanced	Completed
Java	Self	10 Apr 2025	90%	Expert	Completed
Communication	Peer	05 Apr 2025	75%	Intermediate	Completed
SQL	Self	28 Mar 2025	80%	Advanced	Completed
Problem Solving	Self	20 Mar 2025	60%	Intermediate	Completed

My Collection > Get data

GET

http://localhost:8080/api/questions

Docs Params Authorization Headers 8 Body Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL JSON

1

Body Cookies Headers 11 Test Results 1/1

{ } JSON Preview Visualize

```
1  [
2    {
3      "questionId": 1,
4      "assessment": {
5        "assessmentId": 1,
6        "assessmentName": "Java Assessment",
7        "courseName": "Java Programming",
8        "domain": "Backend Development",
9        "totalQuestions": 5,
10       "passingMarks": 3
11     },
12     "questionText": "What is JVM?",
13     "optionA": "Java Virtual Machine",
14     "optionB": "Java Variable Method",
```